Retail Data Engineering Pipeline

...

Link to sign up sheet;
https://docs.google.com/spreadsheets/d/1y4XR7MmZuH96TXVrtfIBZAtUjCwTLEXQZ5F4FjyNeVI/edit?usp=drivesdk

The Scope

Retail Company Scenario

A retail company receives daily raw transaction files for customers, products, and orders. The data contains missing values, inconsistent formatting, duplicate records, and invalid business keys. Before the data can be used it needs to be validated, formatted and loaded into a warehouse.

3 Source Files	ETL Pipeline Pattern	5+ Analytical queries

Project Objectives

ingest structured CSV data from S3
Clean names, data, phone numbers, emails, booleans, currency values, and IDs
Join customers, products and orders into reporting-friendly datasets
Create warehouse ready exports for snowflake.

The Business Problem

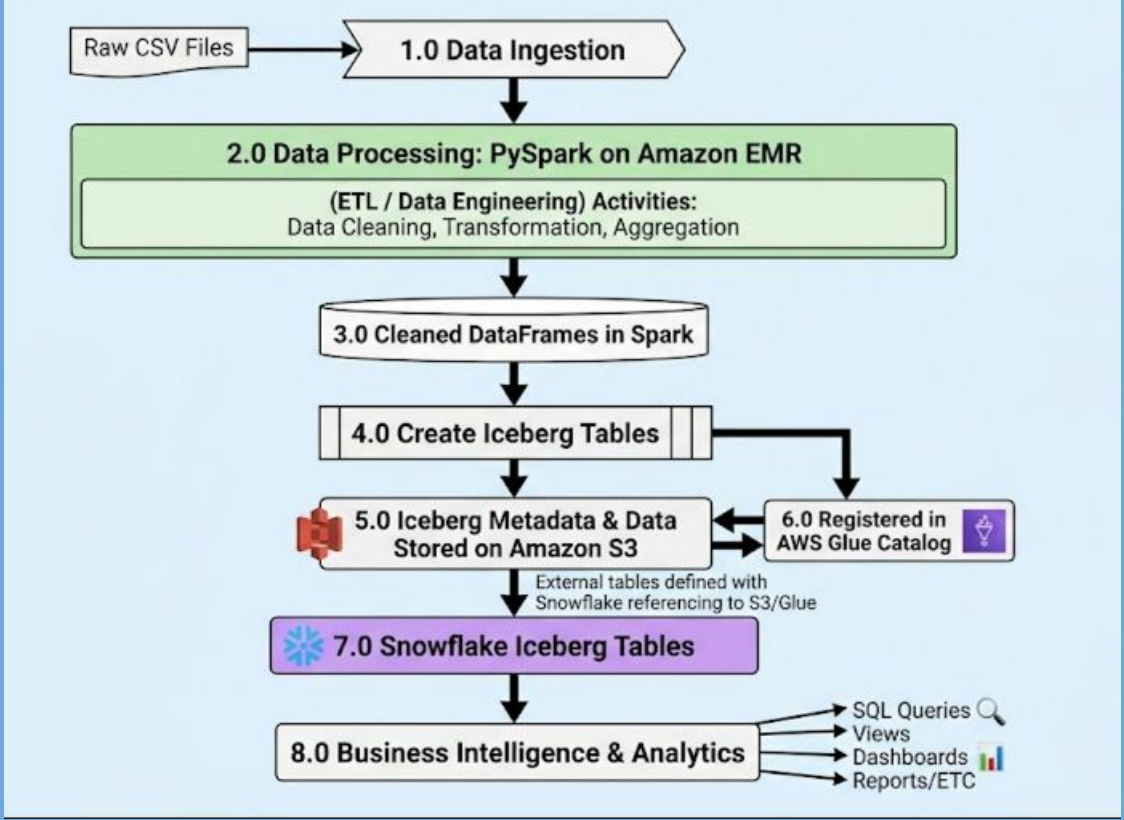
Without a pipeline each report can produce a different answer.

For example duplicate rows can lead to the same customer, product or order appearing more than once.

Mixed formats make dates, currency and the capitalization of text inconsistent.

Invalid relationships may have references to products or orders that do not exist

Solution Architecture




```
    # Specify the S3 warehouse location where Iceberg table data
    # and metadata files will be stored.
    .config(
        "spark.sql.catalog.glue_catalog.warehouse",
        "s3://revature-training-389009238812-us-east-2-an/iceberg/"
    )

    # Configure Iceberg to use the S3FileIO implementation
    # for reading and writing data in Amazon S3.
    .config(
        "spark.sql.catalog.glue_catalog.io-impl",
        "org.apache.iceberg.aws.s3.S3FileIO"
    )
```

```
# No negative or null loyalty points
cleaned_customers = cleaned_customers.withColumn(
    "loyalty_points",
    F.when(
        F.col("loyalty_points").isNull(),
        F.lit(0)
    ).when(
        F.col("loyalty_points") < 0,
        F.lit(0)
    ).otherwise(F.col("loyalty_points"))
)

# If a customer has no id, name, email, or signup date, we drop the record.
cleaned_customers = cleaned_customers.dropna(
    subset=[
        "customer_id",
        "first_name",
        "last_name",
        "email",
        "signup_date"
    ]
)
```

```
CREATE OR REPLACE ICEBERG TABLE order_details
    EXTERNAL_VOLUME = 'project_two_external_volume'
    CATALOG = 'project_two_catalog'
    CATALOG_TABLE_NAME = 'order_details';

-- Analysis Query 1:  Categories by revenue
SELECT
    category,
    SUM(total_amount) AS total_revenue,
    COUNT(order_id) AS number_of_orders
FROM order_details
GROUP BY category
ORDER BY total_revenue DESC;
```

Dataset Overview

Customers Dataset

Role: Core User Data

Represents customer profiles. It stores demographic details, contact information, geographic location, account status (is_active), and loyalty program metrics (loyalty_points).

Products Dataset

Role: Product Catalog Data

Serves as the product catalog. It contains item descriptions, categorization, branding, financial metrics (price, cost), inventory stock levels, physical dimensions (weight), and product status.

Orders Dataset

Role: Transactional Data

Acts as a transactional fact table. It captures individual purchase events linking customers to products, including order and shipping dates, item quantities, pricing, discounts, totals, payment methods, and status.

customers.csv

Some data issues:

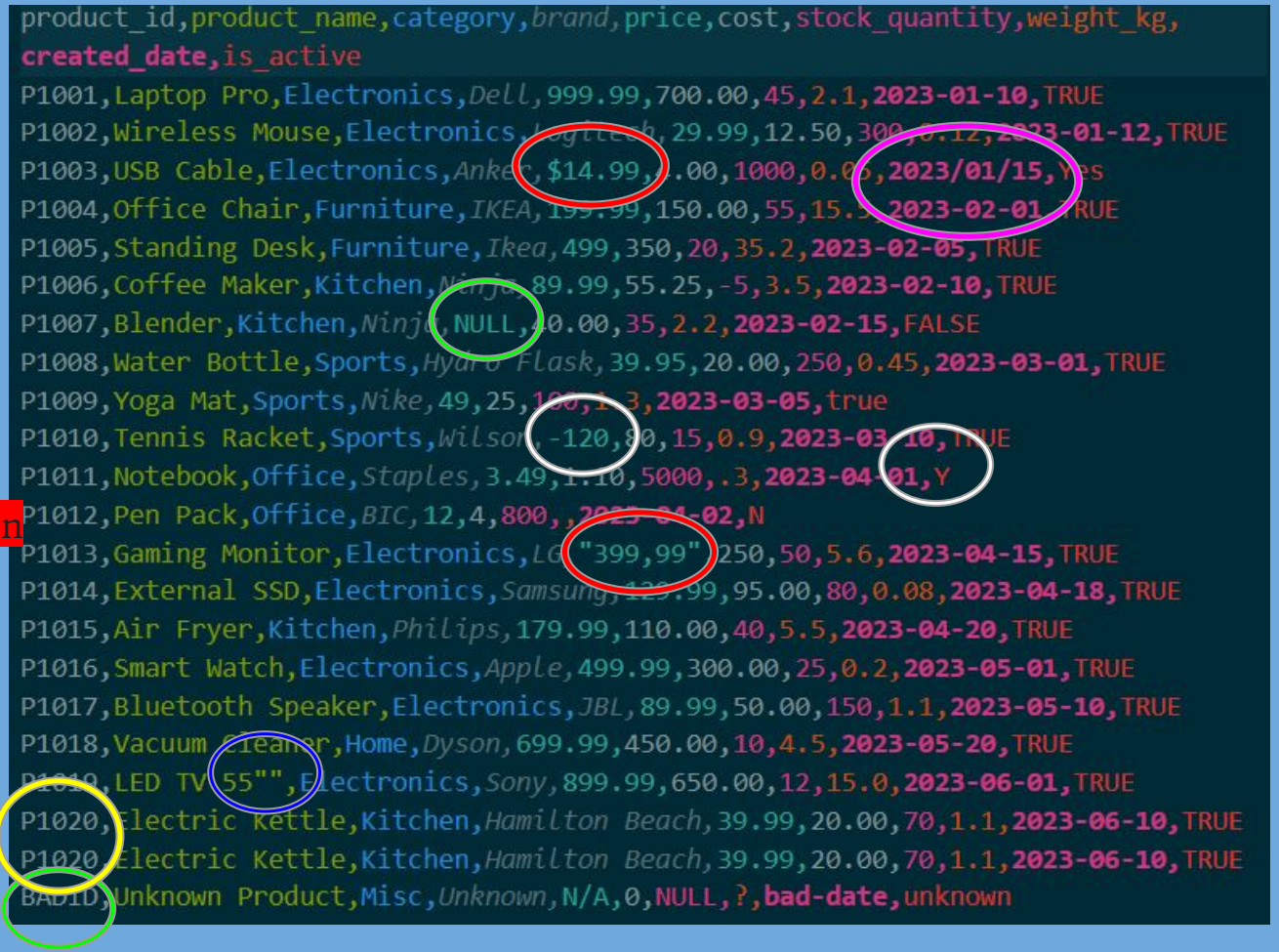
- Incorrect types
- Different formats
- Whitespace
- Invalid entries
- Duplicate entries

```
customer_id,first_name,last_name,email,phone,signup_date,country,state,postal_code,is_active,
loyalty_points
1001,John,Smith,john.smith@gmail.com,6125551001,2023-01-15,USA,MN,55401,TRUE,145
1002,Jane,Doe,jane.doe@yahoo.com,(612)555-2221,2023/02/20,United States,Minnesota,55402,Yes,210
1003,Bob,Johnson,bobjohnsonhotmail.com,6125553331,15-03-2023,US,MN,55403,1,95
1004,Alice,Williams,alice@gmail.com,,2023-04-01,USA,MN,55404,TRUE,
1005,Michael,Brown,michael@@gmail.com,6125555555,2023-04-31,USA,MN,55405,FALSE,310
1006,Sarah,Davis,sarah.davis@gmail.com,612-555-6666,2023-05-14,Canada,ON,M5V3A8,true,510
1007,David,Miller,david@gmail.com,abc123,2023-06-01,USA,WI,53703,false,0
1008,Emily,Wilson,emily@gmail.com,+1 612 555 8888,2023-06-12,USA,MN,55408,Y,85
1009,Chris,Moore,chris@gmail,6125559999,2023-13-01,USA,MN,55409,N,120
1010,Amanda Taylor,amanda@gmail.com,6125551010,2023-07-10,Mexico,CMX,01000,TRUE,75
1011, James , Anderson , james@gmail.com ,6125551011,2023-07-11,USA,MN,55411,TRUE,500
1012,Lisa Thomas,NULL,6125551012,2023-08-15,USA,MN,55412,TRUE,100
1013,Daniel,Jackson,daniel@gmail.com,NULL,2023-08-20,USA,MN,55413,FALSE,90
1014,Jennifer,White,jennifer@gmail.com,6125551014,N/A,USA,MN,55414,TRUE,1000
1015,Matthew,Harris,matthew@gmail.com,6125551015,2023-09-05,USA,MN,,TRUE,-10
1016,Jessica,Martin,jessica@gmail.com,6125551016,2023-09-08,USA,MN,55416,TRUE,999999
1017,Robert,Thompson,robert@gmail.com,6125551017,2023-09-09,USA,mn,55417,true,250
1018,Karen,Garcia,karen@gmail.com,6125551018,2023-09-15,U.S.A.,MN,55418,False,120
1019,Kevin,Martinez,kevin@gmail.com,6125551019,2023-10-01,USA,MN,55419,No,50
1020,Ashley,Robinson,ashley@gmail.com,6125551020,2023-10-11,USA,MN,55420,TRUE,140
1020,Ashley,Robinson,ashley@gmail.com,6125551020,2023-10-11,USA,MN,55420,TRUE,140
XYZ,Invalid,Customer,bademail,555,notadate,USA,MN,ABCDE,maybe,abc
```

products.csv

Some data issues:

- Incorrect types
- Different formats
- Combination of types
- Invalid entries
- Additional characters
- Duplicate entries



orders.csv

Some data issues:

- Incorrect types
- Different formats
- Combination of types
- Duplicate entries
- Missing values

Data Relationships

Customers Dataset

Role: Core User Data

Represents customer profiles. It stores demographic details, contact information, geographic location, account status (is_active), and loyalty program metrics (loyalty_points).

Many to Many

Products Dataset

Role: Product Catalog Data

Serves as the product catalog. It contains item descriptions, categorization, branding, financial metrics (price, cost), inventory stock levels, physical dimensions (weight), and product status.

1

One to Many

Join Table/Fact Table

Orders Dataset

Role: Transactional Data

Acts as a transactional fact table. It captures individual purchase events linking customers to products, including order and shipping dates, item quantities, pricing, discounts, totals, payment methods, and status.

1

One to Many

Foreign key: customer_id

Foreign key: product_id

Extract

Data Ingestion

Where Ingestion Fits in the Pipeline

Reading CSV files from Amazon S3 into PySpark, applying schemas, and validating data types

This step creates the foundation for every cleaning, join, and reporting table after it.

What ingestion does

- Connects Spark to the S3 path
- Reads the three required source files
- Uses schemas instead of guessing
- Produces stable DataFrames for the team

What ingestion does NOT do yet

- It does not calculate revenue
- It does not remove invalid rows
- It does not join tables
- It only prepares clean inputs for those later steps

Goal : safely land raw files into Spark with predictable columns and data types before any cleaning or business transformations happen.

Spark Session for S3 + Iceberg-Ready Processing

The starter project configured Spark for EMR-style cloud processing with Glue Catalog and S3 storage.

```
S3_BUCKET = "s3://####################-us-east-2-an"
RAW_DATA_PATH = S3_BUCKET
ICEBERG_WAREHOUSE_PATH = f"{S3_BUCKET}/iceberg/"
SNOWFLAKE_STAGE_PATH = f"{S3_BUCKET}/snowflake_stage/"
```

```
spark = (
    SparkSession.builder
    .appName("RetailDataPipeline")
    .config(
        "spark.sql.catalog.glue_catalog.warehouse",
        ICEBERG_WAREHOUSE_PATH,
    )
    .config(
        "spark.sql.catalog.glue_catalog.io-impl",
        "org.apache.iceberg.aws.s3.S3FileIO",
```

```
orders_df = (
    spark.read
    .option("header", True)
    .option("mode", "PERMISSIVE")
    .schema(orders_schema)
    .csv(f"{RAW_DATA_PATH}/orders.csv")
)
```

Why this matters

appName
shows job in Spark UI/logs

Iceberg extensions
allow table operations

Glue Catalog
stores table metadata

S3 warehouse
stores data + metadata files

This setup makes Spark ready to read from S3 and later write curated Iceberg tables without changing the ingestion logic.

Applying Schemas While Reading CSV Files

Schemas define the contract: column names, expected data types, and table structure.

```
# The raw columns are intentionally read as strings
# because the files contain
# dirty values such as

orders_schema = StructType([
    StructField("order_id", StringType()),
    StructField("customer_id", StringType()),
    StructField("product_id", StringType()),
    StructField("order_date", StringType()),
    StructField("ship_date", StringType()),
    StructField("quantity", StringType()),
    StructField("unit_price", StringType()),
    StructField("discount_pct", StringType()),
    StructField("total_amount", StringType()),
    StructField("payment_method", StringType()),
    StructField("order_status", StringType()),
])
```

Dataset schemas

customers.csv
customer_id, name, email, phone, signup_date, country, state, active flag

orders.csv
order_id, customer_id, product_id, dates, quantity, price, discount, status

products.csv
product_id, product name, category, brand, price, cost, stock, active flag

Important choice: the completed script reads raw columns as strings first, because dirty files may contain currency symbols, mixed dates, bad booleans, and null-like text. Then the cleaning step safely casts to dates, integers, decimals, and booleans.

Validating Data Types After Ingestion

Quick checks catch bad reads before the pipeline spends time cleaning and joining data.

```
customers_df = (
    spark.read
    .option("header", True)
    .option("mode", "PERMISSIVE")
    .schema(customers_schema)
    .csv(f"{RAW_DATA_PATH}/customers.
)

print("RAW DATA SCHEMAS")
customers_df.printSchema()
products_df.printSchema()
orders_df.printSchema()
```

Validation checklist

1
Schema check
Are column names and data types correct?

2
Record counts
Did all rows load from each file?

3
Sample rows
Do values look like expected retail data?

4
Null review
Did strict casting create unexpected nulls?

Expected source counts in the provided files: customers = 22, products = 22, orders = 24. These counts become the baseline for source-to-target validation later.

Transform

Customer Data Cleaning

Other than simply eliminating duplicate entries and entries with null values, several other techniques were used to clean the data. Every string entry was stripped of whitespace using a single select statement, to reduce bloat.

Next, regex pattern matching was used to eliminate invalid emails and phone numbers, and to strip excess punctuation from country names.

Lastly, state names were capitalized, and negative loyalty points were set to 0.

```
customers_df_clean = customers_df_clean.select([
    func.trim(func.col(c)).alias(c) if t == "string" else func.col(c)
    for c, t in customers_df_clean.dtypes
])
# Regex pattern for a standard email
email_pattern = r"([a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+)"
# Keep only strings that match the email pattern, otherwise return null
customers_df_clean = customers_df_clean.withColumn(
    "email",
    func.when(
        func.col("email").rlike(email_pattern),
        func.col("email")
    )
)
# Ensure loyalty points can't be negative
customers_df_clean = customers_df_clean.withColumn(
    "loyalty_points",
    func.when(func.col("loyalty_points") < 0, 0).otherwise(func.col("loyalty_points"))
)
```

Orders Data Cleaning

- order_date
  - 2023-07-01
  - 2023/07/02
  - bad-date

```
date_with_dashes = F.regexp_replace("order_date", "/", "-")
orders_df_clean = orders_df_clean.withColumn(
    "order_date",
    F.to_date(
        F.try_to_timestamp(date_with_dashes, F.lit("yyyy-MM-dd"))
    )
)
```

- unit_price
  - 29.99
  - $14.99
  - -120
  - “NULL”

```
price = F.regexp_replace(F.col("unit_price"), r"\$", "").try_cast("decimal(10,2)")
orders_df_clean = orders_df_clean.withColumn(
    "unit_price",
    F.when(price >= 0, price)
)
```

- Trimmed spaces

Products Data Cleaning

Main Product Data Issues:

- Dollar signs on “prices”
- Negative values in most numeric columns
- Strings for boolean column “is_active”
- Duplicate rows & NULLS
- Date Format Normalization

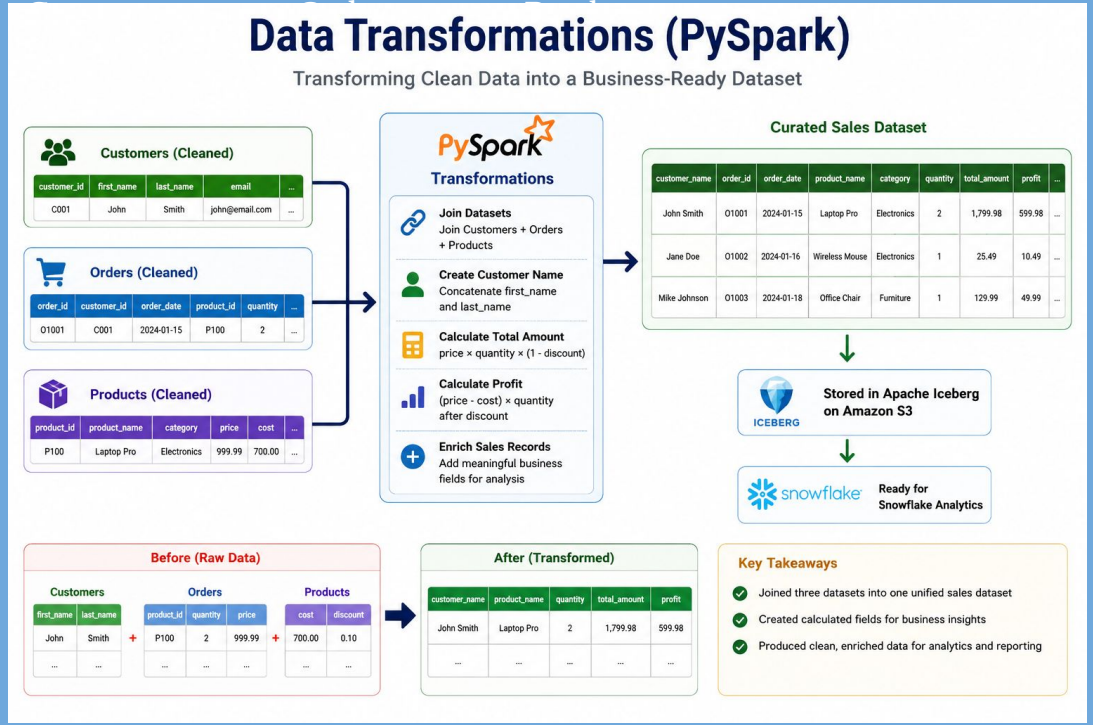
```
.withColumn(
    "price",
    func.regexp_replace(func.col("price"), r"[\$,]", "").try_cast(DecimalType()))
.withColumn(
    "price",
    func.when(func.col("price") < 0, func.lit(0.0)).otherwise(func.col("price")))
.withColumn(
    "price",
    func.round(func.col('price'), func.lit(2))
```

```
.withColumn(
    "stock_quantity",
    func.col('stock_quantity').try_cast('int'))
.withColumn(
    "stock_quantity",
    func.when(func.col("stock_quantity") < 0, func.lit(0))
    .otherwise(func.col("stock_quantity")))
```

```
.withColumn(
    "created_date",
    func.regexp_replace(func.col("created_date"), r"[\/,]", "-"))
.withColumn(
    "created_date",
    func.col('created_date').try_cast('date')
)
```

```
.withColumn(
    "is_active",
    func.when(func.lower(func.trim(func.col("is_active").cast("string"))).rlike("^(true|yes|y|1)$"),func.lit(True))
    .when(func.lower(func.trim(func.col("is_active").cast("string"))).rlike("^(false|no|n|0)$"),func.lit(False))
    .otherwise(func.lit(None)))
```

Data Transformations (PySpark)



PRODUCT_NAME	CATEGORY	BRAND	COST	CUSTOMER_NAME	PROFIT
Vacuum Cleaner	Home	Dyson	450.00	Karen Garcia	214.99
Water Bottle	Sports	Hydro Flask	20.00	Sarah Davis	47.87
Bluetooth Speaker	Electronics	Jbl	50.00	Robert Thompson	79.47
Electric Kettle	Kitchen	Hamilton Beac	20.00	Ashley Robinson	19.99
Laptop Pro	Electronics	Dell	700.00	John Smith	199.99
Led Tv 55""	ics	Sony	650.00	Kevin Martinez	499.98

Workspaces

Aggregations and Summary Metrics

- Sales by payment
  - Which payment method customers prefer?
  - Which payment method processes the most revenue?
- Sales by Category
  - Which category sells the most products?
  - Which category is the most profitable?

```python
# sales by payment method
payment_summary_df = (
    sales_curated_df
    .groupBy("payment_method")
    .agg(
        F.count("*").alias("orders"),
        F.round(F.sum("total_amount"), 2).alias("revenue"),
        F.round(F.sum("profit"), 2).alias("profit")
    )
)
```

```
+--------------+------+-------+------+
|payment_method|orders|revenue|profit|
+--------------+------+-------+------+
|Credit Card   |2     |1564.98|414.98|
|Cash          |1     |39.99  |19.99 |
|PayPal        |1     |229.47 |79.47 |
|MasterCard    |1     |107.87 |47.87 |
+--------------+------+-------+------+
```

```python
category_summary_df = (
    sales_curated_df
    .groupBy("category")
    .agg(
        F.sum("quantity").alias("units_sold"),
        F.round(F.sum("total_amount"), 2).alias("revenue"),
        F.round(F.sum("profit"), 2).alias("profit")
    )
)
```

```
+----------+----------+-------+------+
|category  |units_sold|revenue|profit|
+----------+----------+-------+------+
|Electronics|4         |1129.46|279.46|
|Sports    |3         |107.87 |47.87 |
|Kitchen   |1         |39.99  |19.99 |
|Home      |1         |664.99 |214.99|
+----------+----------+-------+------+
```

Aggregations and Summary Metrics

- Calculates the profit margin percentage for each sale
- Answers which product is the most profitable
- Laptops generate high revenue but low margin

```
margin_df = (
    sales_curated_df
    .select(
        "order_date",
        "product_name",
        "quantity",
        "cost",
        "unit_price",
        "total_amount",
        "profit"
    )
    .withColumn(
        "profit_margin_pct",
        F.round(
            (F.col("profit") / F.col("total_amount")) * 100,
            2
        )
    )
)
```

```
+----------+----------------+--------+------+----------+------------+------+-----------------+
|order_date|product_name    |quantity|cost  |unit_price|total_amount|profit|profit_margin_pct|
+----------+----------------+--------+------+----------+------------+------+-----------------+
|2023-07-20|Electric Kettle |1       |20.00 |39.99     |39.99       |19.99 |49.99            |
|2023-07-06|Water Bottle    |3       |20.00 |39.95     |107.87      |47.87 |44.38            |
|2023-07-17|Bluetooth Speaker|3      |50.00 |89.99     |229.47      |79.47 |34.63            |
|2023-07-18|Vacuum Cleaner  |1       |450.00|699.99    |664.99      |214.99|32.33            |
|2023-07-01|Laptop Pro      |1       |700.00|999.99    |899.99      |199.99|22.22            |
+----------+----------------+--------+------+----------+------------+------+-----------------+
```

Load

Apache Iceberg

```
CREATE OR REPLACE EXTERNAL VOLUME my_external_volume
STORAGE_LOCATIONS =
(
    (
        NAME='s3_Location'
        STORAGE_PROVIDER='S3'
        STORAGE_BASE_URL='s3://link2-bucket-rev-005311909391-us-east-2-an/iceberg/'
        STORAGE_AWS_ROLE_ARN='arn:aws:iam::005311909391:role/SnowflakeIceberg'
    )
)
ALLOW_WRITES = TRUE;
```

Problem:

- When data must be shared across multiple analytic engines, providing interoperability between different processing systems.

Solution:

- Store iceberg data files in central storage (S3 bucket)
- Store metadata in an external catalog (AWS Glue)

```
CREATE OR REPLACE ICEBERG TABLE customers
    EXTERNAL_VOLUME = 'my_external_volume'
    CATALOG = 'my_catalog'
    CATALOG_TABLE_NAME = 'customers';
```

```
CREATE OR REPLACE CATALOG INTEGRATION my_catalog
CATALOG_SOURCE = GLUE
TABLE_FORMAT = ICEBERG
CATALOG_NAMESPACE = 'iceberg_catalog_db'
GLUE_AWS_ROLE_ARN = 'arn:aws:iam::005311909391:role/SnowflakeGlueRole'
GLUE_CATALOG_ID = '005311909391'
GLUE_REGION = 'us-east-2'
ENABLED = TRUE;
```

Snowflake Implementation Summary

- Created a Snowflake warehouse environment for retail analytics using a dedicated database: RETAIL_ANALYTICS_DB
- Organized curated business-ready data in the CURATED schema to separate cleaned analytical datasets from raw inputs
- Configured an External Volume pointing to the Iceberg S3 warehouse location to give Snowflake direct access to S3
- Connected Snowflake to AWS Glue via a Catalog Integration so Snowflake can read Iceberg table metadata without copying data
- Registered curated Iceberg tables directly in Snowflake — no file loading required
- Organized the warehouse to support both operational reporting and higher-level trend analysis

Snowflake Implementation Summary

- Database + schema creation

```
USE ROLE SYSADMIN;
USE WAREHOUSE COMPUTE_WH;

CREATE DATABASE IF NOT EXISTS RETAIL_ANALYTICS_DB;
CREATE SCHEMA IF NOT EXISTS RETAIL_ANALYTICS_DB.CURATED;
```

- External Volume + Catalog Integration

```
CREATE OR REPLACE EXTERNAL VOLUME RETAIL_ICEBERG_EXT_VOL
  STORAGE_LOCATIONS = ((
    NAME = 'PRIMARY_S3'
    STORAGE_PROVIDER = 'S3'
    STORAGE_BASE_URL = 's3://test-bucket-.../iceberg/'
    STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::...:role/SnowflakeIceBerg'
  ));

CREATE OR REPLACE CATALOG INTEGRATION GLUE_RETAIL_CATALOG_INT
  CATALOG_SOURCE = GLUE
  TABLE_FORMAT = ICEBERG
  GLUE_AWS_ROLE_ARN = 'arn:aws:iam::...:role/SnowflakeIceBerg'
  GLUE_CATALOG_ID = '484056256290'
  ENABLED = TRUE;
```

- Iceberg table registration ready for SQL queries

```
CREATE OR REPLACE ICEBERG TABLE SALES_ENRICHED
  EXTERNAL_VOLUME = 'RETAIL_ICEBERG_EXT_VOL'
  CATALOG = 'GLUE_RETAIL_CATALOG_INT'
  CATALOG_NAMESPACE = 'iceberg_catalog_db'
  CATALOG_TABLE_NAME = 'sales_enriched'
  AUTO_REFRESH = TRUE;
```

Analytical SQL

Analytical SQL Queries - Customers That Have Not Ordered

```
SELECT c.customer_id, c.first_name, c.last_name

FROM Customers c

LEFT JOIN  Orders o

        ON c.customer_id = o.customer_id

WHERE o.order_id is NULL;
```

	CUSTOMER_ID	FIRST_NAME	LAST_NAME
1	1004	Alice	Williams
2	1008	Emily	Wilson
3	1011	James	Anderson
4	1013	Daniel	Jackson
5	1015	Matthew	Harris
6	1016	Jessica	Martin

Analytical SQL Queries - Products That Are Not In Any Orders

```
SELECT p.product_id, p.product_name
FROM Products p
LEFT JOIN Orders o
    ON p.product_id = o.product_id
WHERE o.order_id is NULL;
```

	PRODUCT_ID	PRODUCT_NAME
1	P1005	Standing Desk
2	P1009	Yoga Mat
3	P1015	Air Fryer
4	P1016	Smart Watch

Analytical SQL Queries - Total Quantity Sold of Each Product

```
SELECT p.product_id, p.product_name, SUM(o.quantity) AS
total_quantity_sold

FROM Products p

JOIN Orders o

    ON p.product_id = o.product_id

GROUP BY p.product_id, p.product_name

ORDER BY total_quantity_sold DESC;
```

	PRODUCT_ID	PRODUCT_NAME	TOTAL_QUANTITY_SOLD
1	P1017	Bluetooth Speaker	3
2	P1008	Water Bottle	3
3	P1019	LED TV 55	2
4	P1018	Vacuum Cleaner	1
5	P1014	External SSD	1
6	P1010	Tennis Racket	1
7	P1001	Laptop Pro	1
8	P1002	Wireless Mouse	1
9	P1020	Electric Kettle	1

Analytical SQL Queries - Total Sales Per Customers

```
SELECT c.customer_id, c.first_name,
SUM(p.price * o.quantity) AS total_sales
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
JOIN Products p
    ON o.product_id = p.product_id
GROUP BY c.customer_id, c.first_ name
ORDER BY total_sales DESC;
```

# CUSTOMER_ID	A FIRST_NAME	# TOTAL_SALES
1019	Kevin	1799.98
1001	John	999.99
1018	Karen	699.99
1017	Robert	269.97
1006	Sarah	119.85
1020	Ashley	39.99
1010	Amanda	0

Analytical SQL Queries - Total Revenue Per Brand

```
SELECT p.brand, SUM(p.price * o.quantity) AS category_revenue
FROM Products p
JOIN Orders o
    ON p.product_id = o.product_id
GROUP BY p.brand
ORDER BY category_revenue DESC;
```

BRAND	CATEGORY_REVENUE
Sony	1799.98
Dell	999.99
Dyson	699.99
JBL	269.97
Samsung	129.99
Hydro Flask	119.85
Hamilton Beach	39.99
Logitech	29.99
Ninja	0
Wilson	0
IKEA	0

The Big Picture

Data Validation & Quality Assurance

Why do we care?

These validation checks ensure that the data is loaded into Snowflake completely, consistently, and reliably. This improves the accuracy of future reporting and analytics. Essentially, if the data being loaded into Snowflake is garbage, the analytics will be garbage, leading to wasted time and resources.

Challenges and Lessons Learned

LESSONS LEARNED

Defensive ingestion
Always read as strings first, validate, then cast to proper types

Data quality is part of the pipeline
Validate at every stage: raw → clean → Iceberg → Snowflake

Open formats enable interoperability
Write to Iceberg once — PySpark and Snowflake both read it natively

Cloud setup takes as long as the code
EMR, S3, Glue, and IAM config is a significant part of the work

CHALLENGES

Messy real-world data
Currency symbols, 3 date formats, mixed booleans, "NULL" strings

Schema enforcement
Typed schemas silently dropped bad rows — had to read as StringType first

Foreign key issues
Orders referencing customer/product IDs that didn't exist

Connecting Snowflake to S3
IAM trust policy setup required multiple back-and-forth with AWS

Future Improvements

Apache Kafka

Real-time streaming ingestion — replace batch CSV files with live event streams as orders and updates happen

Real-time ingestion

dbt

Business logic transformations inside Snowflake — organized, tested, and documented SQL models for revenue and segmentation

Transformation layer

Power BI

Interactive dashboards connected to Snowflake — business users explore revenue, inventory, and customer trends without writing SQL

BI & analytics

Any Questions?